



Python应用基础

第12章 Pandas数据分析

陈刚

中山大学岭南学院

数据分析篇目录

Contents

第 11 章

NumPy 基础

第 12 章

Pandas 金融数据分析

第 13 章

Matplotlib 数据可视化

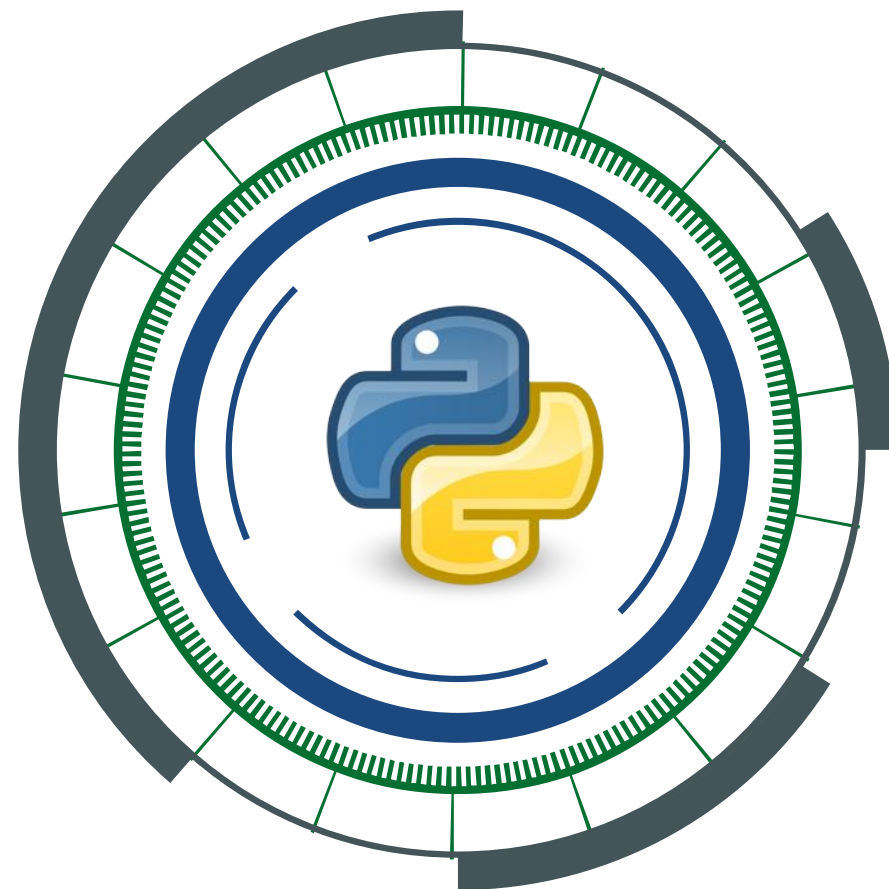


pythonTM
programming



学习目标

- 1 掌握 Series 和 DataFrame 数据结构
- 2 掌握 Pandas 中常用方法
- 3 能够利用 Pandas 进行汇总、计算和数据处理



NumPy 与 Pandas

- 在Python的数据科学世界里，NumPy 和 Pandas 是最坚实的双子星
 - NumPy 负责底层的**高性能数值计算**，而 Pandas 负责上层的**数据分析和处理**
 - **血缘关系**：Pandas 是基于 NumPy 构建的二次开发库。Pandas 的核心数据结构 Series 和 DataFrame，其底层的数据存储和许多数学计算都是由 NumPy 的数组（ndarray）驱动的。
 - **无缝转换**：在 Pandas 中，你可以非常轻松地通过 `.to_numpy()` 或 `.values` 将一个 DataFrame 转换为 NumPy 数组。
 - **兼容性**：NumPy 的绝大多数通用函数（如 `np.sin()`, `np.exp()`）可以直接作用于 Pandas 的对象上。

特性	NumPy	Pandas
主要数据结构	N维数组 (ndarray): 通常是多维的	Series (一维)、 DataFrame (二维表格)
数据类型	单一类型 (Homogeneous): 数组中的所有元素必须是同一种类型 (如全都是 float64 或 int32)	异构类型 (Heterogeneous): 同一张表的不同列可以有不同的数据类型 (如A列是文本, B列是数字, C列是布尔值)
索引方式	仅支持 整数索引 (如0,1,2)	支持 自定义标签索引 (如标签、日期、字符串)
适用场景	科学计算、矩阵运算、图像处理、机器学习底层算法	商业数据分析、清洗脏数据、处理Excel/CSV表格、时间序列分析
内存与性能	内存占用低, 在纯数值矩阵运算上速度极快。	因为附带了索引和丰富的元数据, 内存占用较高, 但提供了更多高阶功能。

NumPy: 纯粹的数学矩阵

关注的是数学结构，像一个没有表头、没有行号的纯数字矩阵

```
import numpy as np

# 创建一个 2x3 的纯数字矩阵
# 注意：如果强行把数字和字符串混在一起，
# NumPy 会把它们全部强制转换为字符串类型！

arr = np.array([[25, "Beijing"], [30, "Shanghai"]])
print(arr)

# 输出:
# [['25' 'Beijing']
#  ['30' 'Shanghai']]
```

Pandas: 全功能的电子表格

关注的是数据业务逻辑，它像一个 Python 版的 Excel 表格，有行索引、有列名，还能容纳不同类型的数据

```
import pandas as pd

# 创建一个 DataFrame，每列的数据类型可以不同

data = {
    "Age": [25, 30],
    "City": ["Beijing", "Shanghai"]
}
df = pd.DataFrame(data)
print(df)

# 输出:
#   Age  City
# 0  25  Beijing
# 1  30  Shanghai
```



选择题：帮小明选工具

- **场景1：**小明要计算 100万个数字的平均值和标准差。用 NumPy 还是 Pandas?
 - **答案：NumPy。**纯数值计算，NumPy 更快更省内存
- **场景2：**小明有一张成绩表，包含姓名、学号、各科成绩、班级。想按班级分组统计平均分。用 NumPy 还是 Pandas?
 - **答案：Pandas。**有标签、有不同类型、需要分组聚合，这是 Pandas 的强项
- **场景3：**小明的数据里有很多空缺，需要填充或删除。用 NumPy 还是 Pandas?
 - **答案：Pandas。**缺失值处理是 Pandas 的核心功能之一

“NumPy 提供高效的数组和数学运算，
Pandas 在它的基础上加了标签索引、缺失值处理和表格操作。
NumPy就像是砖头水泥，而 Pandas 则是用它们盖好的精装修房子！”



12.1

PANDAS的数据结构

概述

- Pandas 中有两种非常重要的数据结构， 分别是 **Series** 和 **DataFrame**。
- **Series** 类似于 NumPy 的**一维数组**， 除了一维数组所用的函数和方法， 还能通过索引进行数据选择， 其索引自动对齐的功能也为计算提供了方便。
- **DataFrame** 类似于 NumPy 的**二维数组**， 同样适用 NumPy 数组的函数和方法。

Series构建方法

- Series 由一组数据（可以是不同数据类型）和与之对应的索引组成。创建 Series 对象可通过向 pd.Series 传递一个列表、字典或一维数组。

```
❶ import numpy as np
❷ s1 = pd.Series([1, 2, 3, np.nan])
❸ s1

0 1.0
1 2.0
2 3.0
3 NaN
dtype: float64

❹ dic = {'浦发银行': 9.72, '民生银行': 5.39, '招商银行': 39.78}
❺ s2 = pd.Series(dic)
❻ s2

浦发银行 9.72
民生银行 5.39
招商银行 39.78
dtype: float64
```

运行一下课本给出的左边的代码！

Series构建方法

- 创建 Series 时，可传入不同类型的一组数据

```
❶ s4 = pd.Series([9.53, 9.62, 9.72])
❷ s4

0 9.53
1 9.62
2 9.72
dtype: float64
```

- 可通过 Series 的 **values** 和 **index** 属性查看对象值和对应的索引

```
❸ s4.values

array([9.53, 9.62, 9.72])

❹ s4.index

RangeIndex(start=0, stop=3, step=1)
```

```
import pandas as pd
import numpy as np
```

```
# 创建一个 Series
dic = {'浦发银行': 9.72, '民生银行': 5.39, '招商银行': 39.78}
s2 = pd.Series(dic)
print("【原始 Series】 : ")
print(s2)
```

```
# 使用 values 和 index 属性查看
print("【values 属性】 只显示值 : ")
print(s2.values)
print("【index 属性】 只显示索引（标签） : ")
print(s2.index)
```

```
【原始 Series】 :
浦发银行    9.72
民生银行    5.39
招商银行   39.78
dtype: float64
【values 属性】 只显示值:
[ 9.72  5.39 39.78]
【index 属性】 只显示索引（标签） :
Index(['浦发银行', '民生银行', '招商银行'], dtype='object')
```

Series构建方法

● 创建 Series 时，也可指定索引

```
❶ s5 = pd.Series([1, np.nan, 7.0, 'abc'], index=['a', 'b', 'c', 'd'])
```

```
❷ s5
```

```
a 1
b NaN
c 7
d abc
dtype: object
```

【情况1】直接打印 Series:

张三 95

李四 87

王五 92

赵六 78

dtype: int64

【情况2】打印 s2.values (只剩数字):

[95 87 92 78]

【情况3】打印 s2.index (只剩名字):

Index(['张三', '李四', '王五', '赵六'], dtype='str')

=====

【思考】以下操作的结果是什么?

成绩的总和 = 352

索引列表 = ['张三', '李四', '王五', '赵六']

```
import pandas as pd
```

```
# 创建 Series
```

```
scores = pd.Series([95, 87, 92, 78], index=['张三', '李四', '王五', '赵六'])
```

```
print(" 【情况1】 直接打印 Series: ")
```

```
print(scores)
```

```
print(" 【情况2】 打印 s2.values (只剩数字) : ")
```

```
print(scores.values)
```

```
print(" 【情况3】 打印 s2.index (只剩名字) : ")
```

```
print(scores.index)
```

```
# 思考题:
```

```
print( "=" * 30)
```

```
print(" 【思考】 以下操作的结果是什么? ")
```

```
print(f"成绩的总和 = {scores.values.sum()}") # 数值求和
```

```
print(f"索引列表 = {list(scores.index)}") # 把索引变成列表
```

Series数据选择

- 与 NumPy 数组不同，Pandas 的 Series 可通过索引选取某个或某些数据

```
❶ s5['a']
```

```
1
```

```
❷ s5[['a', 'b']]
```

```
a 1
```

```
b NaN
```

```
dtype: object
```

```
❸ s5['d'] = 2
```

```
❹ s5
```

```
a 1
```

```
b NaN
```

```
c 7
```

```
d 2
```

```
dtype: object
```

Series数据选择

- Pandas 中的 isnull 和 notnull 函数可以检测缺失数据。

```
❶ dic = {'浦发银行': 9.72, '民生银行': 5.39, '招商银行': 39.78}  
❷ index = ['招商银行', '浦发银行', '民生银行', '工商银行']  
❸ s6 = pd.Series(dic, index)  
❹ pd.isnull(s6)
```

```
招商银行 False  
浦发银行 False  
民生银行 False  
工商银行 True  
dtype: bool
```

● `tolist()`: 把 Pandas/NumPy 的数据结构转换成 Python 原生的列表

```
import pandas as pd
import numpy as np

s = pd.Series([1, 2, 3, np.nan, 5])

print("原始 Series: ")
print(s)

# values 返回的是 NumPy 数组
print(f"\ns.values 的类型: {type(s.values)}")
# index 返回的是 Pandas Index 对象
print(f"s.index 的类型: {type(s.index)}")

# 可以相互转换
print(f"\n把 values 转成列表: {s.values.tolist()}")
print(f"把 index 转成列表: {s.index.tolist()}")
```

原始 Series:

```
0    1.0
1    2.0
2    3.0
3    NaN
4    5.0
```

dtype: float64

s.values 的类型: <class 'numpy.ndarray'>

s.index 的类型: <class 'pandas.RangeIndex'>

把 values 转成列表: [1.0, 2.0, 3.0, nan, 5.0]

把 index 转成列表: [0, 1, 2, 3, 4]

课堂练习1：用Series构建基金收益追踪器

- 小明是理财新手，买了5只基金。他需要用 Python 记录每只基金的**当前收益率**和**持有金额**，并快速找出表现最好和最差的基金
- 任务1（创建数据）
 - 使用字典方式创建一个名为 fund_return 的 Series，记录5只基金的收益率
 - 使用列表 + 指定索引方式创建一个名为 fund_amount 的 Series，记录每只基金的持有金额
 - 打印输出这两个 Series，观察它们的结构
- 任务2（查看数据）
 - 使用 index 属性，获取所有基金名称，并转换为列表打印输出
 - 使用 values 属性，获取所有收益率数据，并打印输出
 - 使用索引标签，单独查看"诺安成长"的收益率
 - 使用 head() 方法，查看前3只基金的持有金额

基金名称	收益率 (%)
招商白酒	12.5
易方达蓝筹	-3.2
中欧医疗	8.7
诺安成长	-15.6
富国天惠	5.4

基金名称	持有金额 (元)
招商白酒	10000
易方达蓝筹	8000
中欧医疗	12000
诺安成长	5000
富国天惠	15000

课堂练习1：用Series构建基金收益追踪器

- 小明是理财新手，买了5只基金。他需要用 Python 记录每只基金的**当前收益率**和**持有金额**，并快速找出表现最好和最差的基金
- 任务3（数据筛选与分析）：
 - 使用布尔索引，筛选出收益率大于 5% 的基金，并打印输出
 - 使用布尔索引，筛选出收益率为负数（即亏损）的基金，并打印输出
 - 计算每只基金的盈利金额，公式为：盈利金额 = 持有金额 × 收益率 ÷ 100。利用 Series 的索引自动对齐特性，一行代码完成计算，并打印结果
 - 使用 max() 和 idxmax() 方法，找出收益率最高的基金及其收益率值

基金名称	收益率 (%)
招商白酒	12.5
易方达蓝筹	-3.2
中欧医疗	8.7
诺安成长	-15.6
富国天惠	5.4

基金名称	持有金额 (元)
招商白酒	10000
易方达蓝筹	8000
中欧医疗	12000
诺安成长	5000
富国天惠	15000

DataFrame构建方法

- DataFrame 是一个表格型数据结构，它由多个列组成，且每列的数据类型可以不同字符串、数值等。

```
❶ dic1 = {'浦发银行': {'2020-10-14': 9.53, '2020-10-15': 9.62, '2020-10-16': 9.72},  
          '民生银行': {'2020-10-14': 5.33, '2020-10-15': 5.35, '2020-10-16': 5.39}}  
❷ df1 = pd.DataFrame(dic1)  
❸ df1
```

	浦发银行	民生银行
2020-10-14	9.53	5.33
2020-10-15	9.62	5.35
2020-10-16	9.72	5.39

```
❹ dic2 = {'浦发银行': [ 9.53, 9.62, 9.72], '民生银行': [ 5.33, 5.35, 5.39]}  
❺ df2 = pd.DataFrame(dic2)  
❻ df2
```

	浦发银行	民生银行
0	9.53	5.33
1	9.62	5.35
2	9.72	5.39

DataFrame构建方法

- 也可以指定列的顺序，这样得到的 DataFrame 就会按照指定方式排列：

```
❶ df3 = pd.DataFrame(dic2, columns=['民生银行', '浦发银行'])  
❷ df3
```

	民生银行	浦发银行
0	5.33	9.53
1	5.35	9.62
2	5.39	9.72

```
❸ df4 = pd.DataFrame(dic2, columns=['民生银行', '浦发银行', '招商银行'])  
❹ df4
```

	民生银行	浦发银行	招商银行
0	5.33	9.53	NaN
1	5.35	9.62	NaN
2	5.39	9.72	NaN

DataFrame构建方法

```
import pandas as pd
```

```
dic1 = {'浦发银行':{'2020-10-14':9.53,'2020-10-15':9.62,'2020-10-16':9.72},  
        '民生银行':{'2020-10-14':5.33,'2020-10-15':5.35,'2020-10-16':5.39}}
```

```
df1 = pd.DataFrame(dic1)
```

```
print(df1)
```

```
dic2 = {'浦发银行':[ 9.53, 9.62, 9.72], '民生银行':[ 5.33, 5.35, 5.39]}
```

```
df2 = pd.DataFrame(dic2)
```

```
print(df2)
```

```
df3 = pd.DataFrame(dic2, columns=['民生银行','浦发银行'])
```

```
print(df3)
```

```
df4 = pd.DataFrame(dic2, columns=['民生银行','浦发银行','招商银行'])
```

```
print(df4)
```

	浦发银行	民生银行
2020-10-14	9.53	5.33
2020-10-15	9.62	5.35
2020-10-16	9.72	5.39

	浦发银行	民生银行
0	9.53	5.33
1	9.62	5.35
2	9.72	5.39

	民生银行	浦发银行
0	5.33	9.53
1	5.35	9.62
2	5.39	9.72

	民生银行	浦发银行	招商银行
0	5.33	9.53	NaN
1	5.35	9.62	NaN
2	5.39	9.72	NaN

DataFrame查看数据

- 利用 head() 或 tail() 方法分别查看 DataFrame 头部和尾部的几行数据:

❶ df5.head(2)

	浦发银行	民生银行
2020-10-14	9.53	5.33
2020-10-15	9.62	5.35

❷ df5.tail(2)

	浦发银行	民生银行
2020-10-15	9.62	5.35
2020-10-16	9.72	5.39

DataFrame查看数据

- 显示索引、列名以及底层的 NumPy 数据：

❶ `df5.index`

`Index(['2020-10-14', '2020-10-15', '2020-10-16'], dtype='object')`

❷ `df5.columns`

`Index(['浦发银行', '民生银行'], dtype='object')`

❸ `df5.values`

`array([[9.53, 5.33],
 [9.62, 5.35],
 [9.72, 5.39]])`

学生成绩单例子： 构建 DataFrame (3种常用方法)

方法1: 通过字典构建 (最常用)

姓名 数学 语文 英语

0 张三 85 90 88

1 李四 92 85 89

2 王五 78 88 85

3 赵六 88 92 91

方法2: 通过字典 + 指定索引

姓名 数学 语文 英语

学生1 张三 85 90 88

学生2 李四 92 85 89

学生3 王五 78 88 85

学生4 赵六 88 92 91

方法3: 通过二维数组构建

数学 语文 英语

张三 85 90 88

李四 92 85 89

王五 78 88 85

赵六 88 92 91

```
import pandas as pd
import numpy as np
print("方法1: 通过字典构建 (最常用)")
data = {
    '姓名': ['张三', '李四', '王五', '赵六'],
    '数学': [85, 92, 78, 88],
    '语文': [90, 85, 88, 92],
    '英语': [88, 89, 85, 91]
}
df1 = pd.DataFrame(data)
print(df1)

print("方法2: 通过字典 + 指定索引")
df2 = pd.DataFrame(data, index=['学生1', '学生2', '学生3', '学生4'])
print(df2)

print("方法3: 通过二维数组构建")
arr = np.array([[85, 90, 88],
                [92, 85, 89],
                [78, 88, 85],
                [88, 92, 91]])
df3 = pd.DataFrame(arr,
                    columns=['数学', '语文', '英语'],
                    index=['张三', '李四', '王五', '赵六'])
print(df3)
```

学生成绩单例子： 查看 DataFrame

```
print("\n" + "=" * 50)
print("查看数据")
print("=" * 50)
```

```
print("【df.head(2)】 查看前2行：")
print(df3.head(2))
```

```
print("\n【df.tail(2)】 查看后2行：")
print(df3.tail(2))
```

```
print("\n【df.info()】 查看基本信息：")
print(df3.info())
```

```
print("\n【df.describe()】 查看统计摘要：")
print(df3.describe())
```

```
print("\n【df.shape】 查看形状（几行几列）：")
print(f"{df3.shape} → 4行3列")
```

```
print("\n【df.columns】 查看列名：")
print(df3.columns.tolist())
```

```
print("\n【df.index】 查看索引：")
print(df3.index.tolist())
```

【df.head(2)】 查看前2行：
数学 语文 英语

张三 85 90 88

李四 92 85 89

【df.tail(2)】 查看后2行：
数学 语文 英语

王五 78 88 85

赵六 88 92 91

【df.info()】 查看基本信息：
<class 'pandas.DataFrame'>

Index: 4 entries, 张三 to 赵六

Data columns (total 3 columns):

Column Non-Null Count Dtype

0 数学 4 non-null int64

1 语文 4 non-null int64

2 英语 4 non-null int64

dtypes: int64(3)

memory usage: 128.0+ bytes

None

【df.describe()】 查看统计摘要：
数学 语文 英语

count 4.000000 4.000000 4.00

mean 85.750000 88.750000 88.25

std 5.909033 2.986079 2.50

min 78.000000 85.000000 85.00

25% 83.250000 87.250000 87.25

50% 86.500000 89.000000 88.50

75% 89.000000 90.500000 89.50

max 92.000000 92.000000 91.00

【df.shape】 查看形状（几行几列）：
(4, 3) → 4行3列

【df.columns】 查看列名：
['数学', '语文', '英语']

【df.index】 查看索引：
['张三', '李四', '王五', '赵六']

DataFrame数据获取

- 如果想从 DataFrame 中获取某一列数据为一个 Series，可以通过类似字典标记或属性的方式：

① `df5['浦发银行']`

```
2020-10-14 9.53  
2020-10-15 9.62  
2020-10-16 9.72  
Name: 浦发银行, dtype: float64
```

② `df5[['浦发银行', '民生银行']]`

	浦发银行	民生银行
2020-10-14	9.53	5.33
2020-10-15	9.62	5.35
2020-10-16	9.72	5.39

③ `df5[df5['浦发银行'] > 9.6]`

	浦发银行	民生银行
2020-10-15	9.62	5.35
2020-10-16	9.72	5.39

DataFrame数据获取

- 如果想从 DataFrame 中获取某一行数据，可以通过位置或名称进行获取，分别使用的索引字段为 `iloc` 和 `loc`:

❶ `df5.iloc[1]`

浦发银行 9.62
民生银行 5.35
Name: 2020-10-15, dtype: float64

❷ `df5.iloc[1:,1:]`

	民生银行
2020-10-15	5.35
2020-10-16	5.39

❸ `df5.iloc[1:,:]`

	浦发银行	民生银行
2020-10-15	9.62	5.35
2020-10-16	9.72	5.39

❹ `df5.loc['2020-10-15']`

浦发银行 9.62
民生银行 5.35
Name: 2020-10-15, dtype: float64

❺ `df5.loc[:,['浦发银行']][1:]`

	浦发银行
2020-10-15	9.62
2020-10-16	9.72

❻ `df5.loc['2020-10-15','浦发银行']`

9.62

❼ `df5[:2]`

	浦发银行	民生银行
2020-10-14	9.53	5.33
2020-10-15	9.62	5.35

❽ `df5[df5['浦发银行']>9.6]`

	浦发银行	民生银行
2020-10-15	9.62	5.35
2020-10-16	9.72	5.39

DataFrame数据获取

- 在 Pandas 中，有多个方法可以选取和重新组合数据，如表所示：

方法	说明
<code>df[val]</code>	从 DataFrame 中选取单列或多列。val 为布尔型数组时，过滤行；val 为切片时，行切片
<code>df.loc[val]</code>	通过 val 选取单行或多行
<code>df.loc[:,val]</code>	通过 val 选取单列或多列
<code>df.loc[val1, val2]</code>	选取 val1 行、val2 列的值
<code>df.iloc[val]</code>	通过整数位置，选取单行或多行
<code>df.iloc[:,val]</code>	通过整数位置，选取单列或多列
<code>df.iloc[val1, val2]</code>	通过整数位置，选取 val1 行、val2 列的值

DataFrame数据获取

操作	语法	返回类型	示例
取一列	df['列名'] 或 df.列名	Series	df['数学']
取多列	df[['列名1', '列名2']]	DataFrame	df[['姓名', '数学']]
取一行（位置）	df.iloc[行号]	Series	df.iloc[0]
取一行（标签）	df.loc['行标签']	Series	df.loc['S1']
取多行（切片）	df[起始:结束]	DataFrame	df[0:3]
取区域（位置）	df.iloc[行范围, 列范围]	DataFrame	df.iloc[0:3, 1:3]
取区域（标签）	df.loc[行标签, 列标签]	DataFrame	df.loc['S1':'S3', '数学':'英语']
取单个值（位置）	df.iloc[行号, 列号]	标量	df.iloc[0, 1]
取单个值（标签）	df.loc['行标签', '列名']	标量	df.loc['S1', '数学']
条件筛选	df[条件表达式]	DataFrame	df[df['数学'] > 85]

DataFrame数据获取：常见坑与技巧

坑1：取单列返回 Series，取多列返回 DataFrame

```
print(type(df['数学'])) # <class 'pandas.core.series.Series'>
```

```
print(type(df[['数学']])) # <class 'pandas.core.frame.DataFrame'>
```

坑2：列名有空格时，不能用 df.列名 的方式

df.学生姓名 **✗** 报错！ 必须用 df['学生姓名']

技巧：链式操作

```
print(df[df['数学'] > 85]['姓名'].tolist()) # 找出数学>85的学生姓名
```

技巧：多个条件要用 &（且）或 |（或），每个条件要加括号

```
df[(df['数学'] > 80) & (df['语文'] > 85)] # 正确
```

```
# df[df['数学'] > 80 and df['语文'] > 85] # ✗ 错误！ 不能用 and
```

DataFrame赋值

- 对DataFrame中某一列的值进行修改，可通过直接赋值一个标量值或一组值：

❶ `df5 = pd.DataFrame({'a': [1, 2, 3], 'b': [5, 6, 7], 'c': [9, 10, 11], 'd': [13, 14, 15]}, index=['x', 'y', 'z'])`

df5

	a	b	c	d
x	1	5	9	13
y	2	6	10	14
z	3	7	11	15

❷ `df5['d'] = 1`
df5

	a	b	c	d
x	1	5	9	1
y	2	6	10	1
z	3	7	11	1

❸ `df5['d'] = np.arange(3)`
df5

	a	b	c	d
x	1	5	9	0
y	2	6	10	1
z	3	7	11	2

DataFrame赋值

- 如果赋值给一个 Series，则会精准匹配对应索引的数值，若 Series 缺失 DataFrame 某些索引，则对应位置为空：

❶ `df5['d'] = pd.Series([2,5,9],index = ['y','z','a'])`
df5

	a	b	c	d
x	1	5	9	NaN
y	2	6	10	2.0
z	3	7	11	5.0

❷ `df5.at[df5.index[0], 'a'] = 0`
df5

	a	b	c	d
x	0	5	9	NaN
y	2	6	10	2.0
z	3	7	11	5.0

❸ `df5.iat[0,3] = 0`
df5

	a	b	c	d
x	0	5	9	0.0
y	2	6	10	2.0
z	3	7	11	5.0

DataFrame赋值

- 通过传递 NumPy 的 array 赋值:

❶ `df5.loc[:, 'a'] = np.array([5] * len(df5))`
`df5`

	a	b	c	d
x	5	5	9	0.0
y	5	6	10	2.0
z	5	7	11	5.0

❷ `df5[df5 > 5] = -df5`
`df5`

	a	b	c	d
x	5	5	-9	0.0
y	5	-6	-10	2.0
z	5	-7	-11	5.0

DataFrame 赋值：基础语法全解析

操作	语法	示例	说明
给一列赋值（标量）	df['列名'] = 值	df['等级'] = 'A'	整列变成同一个值
给一列赋值（列表）	df['列名'] = 列表	df['数学'] = [90,88,85,92,89]	列表长度必须匹配
给一列赋值（Series）	df['列名'] = Series	df['数学'] = s	按索引对齐
给一行赋值	df.loc['行标签'] = 值	df.loc['S1'] = 100	整行变成同一个值
给一行赋值（列表）	df.loc['行标签'] = 列表	df.loc['S1'] = ['张三',90,90,90]	列表长度必须匹配
给一个单元格赋值	df.loc['行','列'] = 值	df.loc['S1','数学'] = 100	最常用
给一个区域赋值	df.loc[行范围, 列范围] = 值	df.loc['S1':'S3','数学':'英语'] = 0	区域批量赋值
新增列	df['新列名'] = 值	df['总分'] = df.sum(axis=1)	列名不存在时自动新增

课堂练习2：卷王班成绩管理

- 创建一个 DataFrame 存储班级成绩：
 1. 数据包含5名学生：'张三','李四','王五','赵六','钱七'
 2. 包含3门科目成绩：'Python','Pandas','SQL'
 3. 成绩数据如表
 4. 将姓名列设置为行索引
 5. 打印输出整个 DataFrame
- 查看数据：
 1. 使用 head() 查看前3行数据
 2. 使用 tail() 查看后2行数据
 3. 使用 info() 查看 DataFrame 的基本信息
 4. 使用 describe() 查看成绩的统计摘要
 5. 打印输出：共有多少名学生？共多少门科目？

姓名	Python	Pandas	SQL
张三	85	90	88
李四	92	85	89
王五	78	88	85
赵六	88	92	91
钱七	95	89	93

课堂练习2：卷王班成绩管理

- 使用 loc 和 iloc 完成数据获取：

1. 获取"李四"的全部成绩（使用行标签）
2. 获取第3行（王五）的全部成绩（使用位置索引）
3. 获取所有学生的 Pandas 成绩（一列）
4. 同时获取 Python 和 SQL 两列成绩
5. 获取 赵六 的 Pandas 成绩（单个值）
6. 获取前3行、后2列的数据区域（Python 和 Pandas 列）
7. 筛选出 Python 成绩大于 90 分的学生

- 数据赋值：

1. 新增一列 '总分'，计算每位学生三门成绩的总和
2. 新增一列 '平均分'，计算每位学生的平均分（保留1位小数）
3. 将 王五 的 Python 成绩从 78 修改为 82
4. 将 赵六 的全部成绩修改为 [90, 90, 90]
5. 将 Pandas 成绩低于 90 分的学生的 Pandas 成绩全部加 5 分（但不能超过100）
6. 在 姓名 列后面插入一列 '班级'，所有学生统一赋值为 '一班'

姓名	Python	Pandas	SQL
张三	85	90	88
李四	92	85	89
王五	78	88	85
赵六	88	92	91
钱七	95	89	93



12.2

PANDAS 的常用方法

索引对象

- 构造 Series 或 DataFrame 时，所用到的任何数组或序列标签都会被转换成为一个 Index。

```
❶ obj = pd.Series(range(3), index=['a', 'b', 'c'])
❷ obj.index
Index(['a', 'b', 'c'], dtype='object')
```

- 下表列举了 Index 对象的方法和属性，并给出了实例说明。

方法	说明
append	连接两个 Index 对象，产生新 Index
intersection	计算交集
union	计算并集
isin	判断一个 Index 的各值是否在另一个 Index 中，返回一个布尔值数组
delete	传入位置参数 i，删除索引 i 处的元素，得到新 Index
drop	传入索引值 i，删除该索引，得到新 Index
insert	传入位置参数 i 和索引值 j，将索引 j 插入第 i 的位置，返回一个新 Index
unique	计算 Index 中唯一值的数组

```
❶ obj2 = pd.Series(range(4), index=['a', 'c', 'd', 'e'])
❷ obj.index.append(obj2.index)
Index(['a', 'b', 'c', 'a', 'c', 'd', 'e'], dtype='object')
```

索引对象 Index

- Index 就像是数据的“身份证标签”，它给每一行、每一列起了一个名字

```
idx1: Index(['A', 'B', 'C', 'D'], dtype='str')
idx2: Index(['C', 'D', 'E', 'F'], dtype='str')
交集: Index(['C', 'D'], dtype='str')
并集: Index(['A', 'B', 'C', 'D', 'E', 'F'], dtype='str')
'B' 是否在 idx1 中? [False True False False]
idx1 中的元素在 idx2 中吗? [False False True True]
删除索引位置1后的结果: Index(['A', 'C', 'D'], dtype='str')
删除 'C' 后的结果: Index(['A', 'B', 'D'], dtype='str')
在位置2插入 'X' 后的结果: Index(['A', 'B', 'X', 'C', 'D'], dtype='str')
去重后的索引: Index(['A', 'B', 'C'], dtype='str')
```

```
import pandas as pd
```

```
# 创建两个索引
```

```
idx1 = pd.Index(['A', 'B', 'C', 'D'])
```

```
idx2 = pd.Index(['C', 'D', 'E', 'F'])
```

```
print("idx1:", idx1)
```

```
print("idx2:", idx2)
```

```
# 1. 交集 (两个索引都有的)
```

```
print("交集: ", idx1.intersection(idx2))
```

```
# 2. 并集 (两个索引合起来)
```

```
print("并集: ", idx1.union(idx2))
```

```
# 3. 判断元素是否在索引中
```

```
print("'B' 是否在 idx1 中? ", idx1.isin(['B']))
```

```
print("idx1 中的元素在 idx2 中吗? ", idx1.isin(idx2))
```

```
# 4. 删除指定位置的索引
```

```
print("删除索引位置1后的结果: ", idx1.delete(1))
```

```
# 5. 删除指定值的索引
```

```
print("删除 'C' 后的结果: ", idx1.drop('C'))
```

```
# 6. 插入新索引
```

```
print("在位置2插入 'X' 后的结果: ", idx1.insert(2, 'X'))
```

```
# 7. 去重
```

```
idx3 = pd.Index(['A', 'A', 'B', 'C', 'C'])
```

```
print("去重后的索引: ", idx3.unique())
```

重新索引

- Reindex: 允许修改、增加、删除指定轴上的索引，并返回一个数据副本。

```
❶ obj = pd.Series([39.78, 9.72, 5.39], index = ['招商银行', '浦发银行', '民生银行'])  
❷ obj
```

```
招商银行 39.78  
浦发银行 9.72  
民生银行 5.39  
dtype: float64
```

```
❸ obj.reindex(['浦发银行', '工商银行', '招商银行', '民生银行'], fill_value=0)
```

```
浦发银行 9.72  
工商银行 0.00  
招商银行 39.78  
民生银行 5.39  
dtype: float64
```

重新索引

- 对于时间序列数据，在重新索引时很可能出现空值，这时可能需要进行插值处理，method 选项可实现插值，ffill 是前向填充，bfill 是后向填充。

```
❶ obj = pd.Series(['a', 'b', 'c'], index=[0, 2, 3])  
❷ obj.reindex(np.arange(5), method='ffill')
```

```
0 a  
1 a  
2 b  
3 c  
4 c  
dtype: object
```

重新索引

- 对于 DataFrame 来说，reindex 既可以修改行索引，也可以修改列索引。如果只传入一个序列，则会重新索引行，使用 columns 关键字可重新索引列。

```
① df = pd.DataFrame(np.arange(9).reshape(3,3), index=['a', 'c', 'b'], columns=['x', 'y', 'z'])  
② df
```

	x	y	z
a	0	1	2
c	3	4	5
b	6	7	8

```
③ df.reindex(['a', 'b', 'c', 'd'])
```

	x	y	z
a	0.0	1.0	2.0
b	6.0	7.0	8.0
c	3.0	4.0	5.0
d	NaN	NaN	NaN

```
④ df.reindex(columns=['y', 'z', 'x'])
```

	y	z	x
a	1	2	0
c	4	5	3
b	7	8	6

重新索引 reindex

- reindex 就像“重新排列座位号”，可以调整顺序、增加新座位、删除旧座位，缺的座位自动空着

```
import pandas as pd
import numpy as np

# 原始数据
s = pd.Series([10, 20, 30], index=['A', 'B', 'C'])
print("原始 Series: ")
print(s)

# 重新索引：调整顺序 + 增加新索引
print("\n重新索引为 ['C', 'B', 'A', 'D']: ")
print(s.reindex(['C', 'B', 'A', 'D']))

# 用 fill_value 填充缺失值
print("\n用 0 填充缺失值: ")
print(s.reindex(['C', 'B', 'A', 'D'], fill_value=0))
```

原始 Series:

A 10

B 20

C 30

dtype: int64

重新索引为 ['C', 'B', 'A', 'D']:

C 30.0

B 20.0

A 10.0

D NaN

dtype: float64

用 0 填充缺失值:

C 30

B 20

A 10

D 0

dtype: int64

- 时间序列的 ffill / bfill (插值填充)

```
import pandas as pd
import numpy as np

# 时间序列数据
dates = pd.date_range('2024-01-01', periods=5, freq='D')
s = pd.Series([10, 20, np.nan, 40, 50], index=dates)
print("原始时间序列: ")
print(s)
# 重新索引到更密集的时间点
new_dates = pd.date_range('2024-01-01', periods=9, freq='D')
print("\n重新索引到每天: ")
print(s.reindex(new_dates))
# ffill: 用前面的值填充后面的空值
print("\n使用 ffill (前向填充): ")
print(s.reindex(new_dates, method='ffill'))
# bfill: 用后面的值填充前面的空值
print("\n使用 bfill (后向填充): ")
print(s.reindex(new_dates, method='bfill'))
```

使用 ffill (前向填充) :

2024-01-01	10.0
2024-01-02	20.0
2024-01-03	NaN
2024-01-04	40.0
2024-01-05	50.0
2024-01-06	50.0
2024-01-07	50.0
2024-01-08	50.0
2024-01-09	50.0

Freq: D, dtype: float64

使用 bfill (后向填充) :

2024-01-01	10.0
2024-01-02	20.0
2024-01-03	NaN
2024-01-04	40.0
2024-01-05	50.0
2024-01-06	NaN
2024-01-07	NaN
2024-01-08	NaN
2024-01-09	NaN

Freq: D, dtype: float64

- DataFrame 的 reindex (行和列)

```
import pandas as pd
import numpy as np
# 创建 DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
}, index=['x', 'y', 'z'])
print("原始 DataFrame: ")
print(df)
# 只传入序列: 重新索引行
print("\n重新索引行 ['z', 'x', 'w']: ")
print(df.reindex(['z', 'x', 'w']))
# 使用 columns 关键字: 重新索引列
print("\n重新索引列 ['C', 'A', 'D']: ")
print(df.reindex(columns=['C', 'A', 'D']))
# 同时重新索引行和列
print("\n同时重新索引行和列: ")
print(df.reindex(index=['z', 'y', 'w'], columns=['C', 'B', 'D']))
```

原始 DataFrame:

	A	B	C
x	1	4	7
y	2	5	8
z	3	6	9

重新索引行 ['z', 'x', 'w']:

	A	B	C
z	3.0	6.0	9.0
x	1.0	4.0	7.0
w	NaN	NaN	NaN

重新索引列 ['C', 'A', 'D']:

	C	A	D
x	7	1	NaN
y	8	2	NaN
z	9	3	NaN

同时重新索引行和列:

	C	B	D
z	9.0	6.0	NaN
y	8.0	5.0	NaN
w	NaN	NaN	NaN

重新索引

● `reindex` 函数的参数和意义:

方法	意义
<code>index</code>	用于重新索引的序列顺序
<code>method</code>	插值填充的方法, <code>ffill</code> 或 <code>bfill</code>
<code>fill_value</code>	用于对缺失值填充数值
<code>limit</code>	前向或后向填充的最大填充量
<code>level</code>	在 <code>MultilIndex</code> 的指定级别上匹配简单索引, 否则选取其子集
<code>copy</code>	默认为 <code>True</code> , 无论如何都复制, 若改为 <code>False</code> 则新旧相等就不复制

参数	含义	示例
<code>index</code>	新行索引序列	<code>reindex(['a','b','c'])</code>
<code>columns</code>	新列索引序列	<code>reindex(columns=['x','y'])</code>
<code>method</code>	插值方法: <code>'ffill'</code> (前向)、 <code>'bfill'</code> (后向)	<code>reindex(method='ffill')</code>
<code>fill_value</code>	缺失值填充值	<code>reindex(fill_value=0)</code>
<code>limit</code>	前向/后向填充的最大数量	<code>reindex(method='ffill', limit=1)</code>
<code>copy</code>	是否返回新对象, 默认 <code>True</code>	<code>reindex(copy=False)</code>

删除指定轴上的项重新索引

- 想要删除某轴上的项，可以采用 drop 方法，传入一个索引数组或列表即可，结果会返回一个在指定轴上删除了指定项的新对象。

❶ `df.drop('a')`

	x	y	z
c	3	4	5
b	6	7	8

❷ `df.drop(['a','c'])`

	x	y	z
b	6	7	8

❸ `df.drop('y',axis=1)`

	x	z
a	0	2
c	3	5
b	6	8

注意：

- Index 是一个类 (Class)
- reindex 是一个方法 (Method)
- 它们遵循不同的命名规范

```
import pandas as pd
# Index —— 这是一个类 (Class) , 首字母大写
print(pd.Index)
# 输出: <class 'pandas.core.indexes.base.Index'>

# reindex —— 这是一个方法 (Method) , 全小写
df = pd.DataFrame({'A': [1, 2, 3]})
print(df.reindex)
# 输出: <bound method DataFrame.reindex of ...>

# 调用时的区别
# Index 是类, 用来创建对象
idx = pd.Index(['a', 'b', 'c']) # ✅ 正确
# idx = pd.index(['a', 'b', 'c']) # ❌ 错误! index 全小写不存在
# reindex 是方法, 用来调用
df.reindex([0, 2, 1]) # ✅ 正确
# df.Reindex([0, 2, 1]) # ❌ 错误! Reindex 首字母大写不存在
```

名称	类型	命名规范	示例
类 (Class)	对象的"蓝图"	首字母大写	Index, Series, DataFrame
方法 (Method)	类的"功能"	全小写, 单词间用下划线	reindex(), drop(), head()
函数 (Function)	独立功能	全小写, 单词间用下划线	pd.read_csv(), np.mean()
变量 (Variable)	存储数据	全小写, 单词间用下划线	df, my_data, student_name

排序

- 对行或列索引进行排序，可用 `sort_index` 方法。
对 DataFrame 的一个或多个列的值进行排序，可用 `sort_values` 并将一个或多个列的名字传递给 `by` 选项。

❶ `df5.sort_index(axis=1, ascending=False)`

	d	c	b	a
x	0.0	-9	5	5
y	2.0	-10	-6	5
z	5.0	-11	-7	5

❷ `obj = Series([4,2,9,-1])`
`obj.sort_index()`

```
0 4
1 2
2 9
3 -1
dtype: int64
```

❸ `df = DataFrame({'b': [3,-1,-1,0],`
`'a': [1,9,-6,4]})`
`df.sort_values(by='b')`

	b	a
1	-1	9
2	-1	-6
3	0	4
0	3	1

排序 sort_index 与 sort_values

- sort_index 排"名字", sort_values 排"分数"

方法	作用	排序依据
sort_index()	按索引排序	按行索引或列索引的值排序
sort_values()	按值排序	按某一行或多列的数据值排序

- 语法

- axis 决定"沿着哪个方向"操作
- ascending 决定"从小到大"还是"从大到小"
- inplace 决定"直接改原表"还是"复制一份再改"

```
df.sort_index(axis=0, ascending=True, inplace=False)
df.sort_values(by='列名', ascending=True, inplace=False)
```

axis 值	含义	通俗理解
axis=0 或 axis='index'	按行操作	上下方向, 处理每一列的数据
axis=1 或 axis='columns'	按列操作	左右方向, 处理每一行的数据

ascending 值	含义	通俗理解
ascending=True (默认)	升序	从小到大, 从低到高
ascending=False	降序	从大到小, 从高到低

inplace 值	含义	效果
inplace=False (默认)	返回新对象	原数据不变, 产生新的 DataFrame
inplace=True	原地修改	直接修改原数据, 不返回新对象

```
import pandas as pd
```

```
# 创建乱序数据
```

```
df = pd.DataFrame({  
    '姓名': ['张三', '李四', '王五', '赵六'],  
    '成绩': [85, 92, 78, 88]  
}, index=[3, 1, 4, 2])
```

```
print("原始数据（索引乱序）：")  
print(df)
```

```
print("\n按行索引升序排序：")  
print(df.sort_index())
```

```
print("\n按行索引降序排序：")  
print(df.sort_index(ascending=False))
```

```
print("\n按列索引排序（按列名排序）：")  
df2 = pd.DataFrame({'B': [1,2], 'A': [3,4], 'C': [5,6]})  
print(df2)  
print("\n按列名排序：")  
print(df2.sort_index(axis=1))
```

```
import pandas as pd
```

```
df = pd.DataFrame({  
    '姓名': ['张三', '李四', '王五', '赵六', '钱七'],  
    '班级': ['一班', '二班', '一班', '二班', '一班'],  
    '成绩': [85, 92, 78, 88, 95]  
})
```

```
print("原始数据：")  
print(df)
```

```
print("\n按成绩升序排序（低到高）：")  
print(df.sort_values(by='成绩'))
```

```
print("\n按成绩降序排序（高到低）：")  
print(df.sort_values(by='成绩', ascending=False))
```

```
print("\n按班级排序，同班内按成绩排序：")  
print(df.sort_values(by=['班级', '成绩']))
```

```
print("\n按班级升序，成绩降序：")  
print(df.sort_values(by=['班级', '成绩'], ascending=[True, False]))
```

算术运算和数据对齐

- Series 的数据运算（布尔值数据过滤、标量乘法、函数）会保留索引和值之间的对应关系。

```
❶ obj = pd.Series([1,3,-2,4],index=['a','b','c','d'])  
obj
```

```
a 1  
b 3  
c -2  
d 4  
dtype: int64
```

```
❷ obj[obj>0]
```

```
a 1  
b 3  
d 4  
dtype: int64
```

```
❸ obj * obj
```

```
a 1  
b 9  
c 4  
d 16  
dtype: int64
```

```
❹ np.log(obj[obj>0])
```

```
a 0.000000  
b 1.098612  
d 1.386294 dtype: float64
```

算术运算和数据对齐

- DataFrame 会同时在行和列上对齐再运算。

```
① df1 = pd.DataFrame(np.arange(6).reshape(2,3), index=['a', 'b'], columns=['x', 'y', 'z'])  
df2 = pd.DataFrame(np.arange(9).reshape(3,3), index=['a', 'b', 'c'], columns =['x', 'y', 'z2'])
```

	x	y	z
a	0	1	2
b	3	4	5

	x	y	z2
a	0	1	2
b	3	4	5
c	6	7	8

```
② df1 + df2
```

	x	y	z	z2
a	0.0	2.0	NaN	NaN
b	6.0	8.0	NaN	NaN
c	NaN	NaN	NaN	NaN

```
③ df1.add(df2, fill_value=0)
```

	x	y	z	z2
a	0.0	2.0	2.0	2.0
b	6.0	8.0	5.0	5.0
c	6.0	7.0	NaN	8.0

算术运算和数据对齐

- 如果某个索引值在 DataFrame 的列或 Series 的索引中找不到，则参与运算的两个对象就会被重新索引形成并集。

```
❶ series = pd.Series(np.arange(4), index=['x', 'y', 'm', 'n'])  
series
```

```
x 0  
y 1  
m 2  
n 3  
dtype: int64
```

```
❷ df1 + series
```

	m	n	x	y	z
a	NaN	NaN	0.0	2.0	NaN
b	NaN	NaN	3.0	5.0	NaN

算术运算和数据对齐

- 如果需要匹配行，然后在列上传播，则需要使用算术运算方法。

```
❶ series2 = df1['x']  
series2
```

a 0

b 3

Name: x, dtype: int64

```
❷ df1.add(series2, axis=0)
```

	x	y	z
a	0	1	2
b	6	7	8

缺失值检测和处理

● 数据缺失是一个普遍存在的现象。Pandas 设计了一系列方法来减小缺失数据对数据处理的影响。缺失值的处理方法如下表所示。

方法	意义
dropna	删除所有缺失值所在轴的数据，0 表示对行删除，1 表示对列删除
fillna	用指定值或插值方法填充缺失数据，如 ffill 或 bfill
isnull	判断哪些数值是缺失值，返回一个布尔值的对象
notnull	isnull 的否定式

方法	作用	返回值类型	通俗理解
isnull()	检测哪些是缺失值	bool (True=缺失)	"扫描空位"
notnull()	检测哪些不是缺失值	bool (True=非缺失)	"扫描非空位"
dropna()	删除缺失值所在的行/列	DataFrame/Series	"扔掉有空位的行"
fillna()	用指定值填充缺失值	DataFrame/Series	"把空位填上"

缺失值检测： isnull()与 notnull()

```
import pandas as pd
import numpy as np
```

```
df = pd.DataFrame({
    'A': [1, np.nan, 3],
    'B': [np.nan, 5, 6]
})
```

```
print("原始数据：")
print(df)
```

```
print("\nisnull(): 哪些是缺失值？")
print(df.isnull())
```

```
print("\nnotnull(): 哪些不是缺失值？")
print(df.notnull())
```

```
# 常用：统计每列有多少缺失值
print("\n每列缺失值数量：")
print(df.isnull().sum())
```

```
# 常用：筛选出没有缺失值的行
print("\n筛选出所有列都没有缺失值的行：")
print(df[df.notnull().all(axis=1)])
```

原始数据：

	A	B
0	1.0	NaN
1	NaN	5.0
2	3.0	6.0

isnull(): 哪些是缺失值？

	A	B
0	False	True
1	True	False
2	False	False

notnull(): 哪些不是缺失值？

	A	B
0	True	False
1	False	True
2	True	True

每列缺失值数量：

A	1
B	1

dtype: int64

筛选出所有列都没有缺失值的行：

	A	B
2	3.0	6.0

缺失值删除

- 如何过滤缺失值是数据预处理的重要环节，Pandas 提供的 dropna 方法非常实用。对于 Series 而言，dropna 返回一个仅含非空数据和索引值的 Series。

```
❶ series = pd.Series({'浦发银行': 9.72, '民生银行': 5.39, '招商银行': 39.78, '工商银行': np.nan})  
series
```

```
浦发银行 9.72  
民生银行 5.39  
招商银行 39.78  
工商银行 NaN  
dtype: float64
```

```
❷ series.dropna()
```

```
浦发银行 9.72  
民生银行 5.39  
招商银行 39.78  
dtype: float64
```

```
❸ series[series.notnull()]
```

```
浦发银行 9.72  
民生银行 5.39  
招商银行 39.78  
dtype: float64
```

缺失值删除

- 对于 DataFrame 而言，dropna 方法默认将含有缺失值的行或列（也包括全部为缺失值的轴）丢弃。

❶ `df = pd.DataFrame([[1,3,5],[2,np.nan,6],[np.nan]*3])`
`df`

	0	1	2
0	1.0	3.0	5.0
1	2.0	NaN	6.0
2	NaN	NaN	NaN

❷ `df.dropna()`

	0	1	2
0	1.0	3.0	5.0

❸ `df.dropna(how='all')`

	0	1	2
0	1.0	3.0	5.0
1	2.0	NaN	6.0

缺失值填充

- 填补缺失值多数情况下 fillna 是我们主要运用的函数。

❶ `df.fillna(value=0)`

	0	1	2
0	1.0	3.0	5.0
1	2.0	0.0	6.0
2	0.0	0.0	0.0

❷ `df.fillna({0:0,1:1,2:1.5})`

	0	1	2
0	1.0	3.0	5.0
1	2.0	1.0	6.0
2	0.0	1.0	1.5

❸ `df.fillna({0:0,1:1,2:1.5},inplace=True)`
`df`

	0	1	2
0	1.0	3.0	5.0
1	2.0	1.0	6.0
2	0.0	1.0	1.5

课堂练习3：成绩表缺失处理

某班级考试成绩表存在缺考记录，请用Pandas 缺失值处理方法完成以下任务：

1. 用 `isnull()` 找出数据表哪些位置是缺失值
2. 用 `dropna()` 删除任何有缺考科目的学生记录
3. 用 `fillna(0)` 将缺考成绩按 0 分 填充
4. 用 `fillna(df.mean())` 将缺考成绩按各科平均分填充
5. 用 `isnull().sum()` 统计每门科目的缺考人数

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    '姓名': ['张三', '李四', '王五', '赵六', '钱七'],
    '数学': [85, np.nan, 78, 88, 95],
    '语文': [90, 85, np.nan, 92, 89],
    '英语': [88, 89, 85, np.nan, 93]
})
```

课堂练习3：成绩表缺失处理

某班级考试成绩表存在缺考记录，请用Pandas 缺失值处理方法完成以下任务：

1. 用 `isnull()` 找出数据表哪些位置是缺失值
2. 用 `dropna()` 删除任何有缺考科目的学生记录
3. 用 `fillna(0)` 将缺考成绩按 0 分 填充
4. 用 `fillna(df.mean())` 将缺考成绩按各科平均分填充
5. 用 `isnull().sum()` 统计每门科目的缺考人数

预期输出参考右图：

任务1：缺失值位置

	姓名	数学	语文	英语
0	False	False	False	False
1	False	True	False	False
2	False	False	True	False
3	False	False	False	True
4	False	False	False	False

任务2：删除有缺考的学生

	姓名	数学	语文	英语
0	张三	85.0	90.0	88.0
4	钱七	95.0	89.0	93.0

任务3：缺考按0分填充

	姓名	数学	语文	英语
0	张三	85.0	90.0	88.0
1	李四	0.0	85.0	89.0
2	王五	78.0	0.0	85.0
3	赵六	88.0	92.0	0.0
4	钱七	95.0	89.0	93.0

任务4：缺考按平均分填充

	姓名	数学	语文	英语
0	张三	85.0	90.0	88.00
1	李四	86.5	85.0	89.00
2	王五	78.0	89.0	85.00
3	赵六	88.0	92.0	88.75
4	钱七	95.0	89.0	93.00

任务5：各科缺考人数

```
数学    1
语文    1
英语    1
dtype: int64
```

apply 函数

- apply 方法能快速完成应用一个函数于 DataFrame 的某一列

❶ `df = pd.DataFrame([[1,2,3],[5,3,7]])`
`df`

	0	1	2
0	1	2	3
1	5	3	7

准备数据

❷ `df.apply(np.cumsum)`

	0	1	2
0	1	2	3
1	6	5	10

`np.cumsum` 是 NumPy 的累加函数

`apply(np.cumsum)` 表示：对每一列应用累加

❸ `df.apply(lambda x:x.max()-x.min())`

0 4

1 1

2 4

`dtype: int64`

参数 `x` 代表每一列数据

`x.max()` 是该列的最大值

`x.min()` 是该列的最小值

结果：每列的极差（最大值与最小值的差）

❹ `def f(x):`
 `return pd.Series([x.sum(), x.mean()],`
 `index=['sum', 'mean'])`
`df.apply(f)`

	0	1	2
sum	6.0	5.0	10.0
mean	3.0	2.5	5.0

自定义函数 `f(x)` 接收一列数据 `x`

返回一个 Series，包含两个值：

`apply(f)` 对每一列执行这个函数，
然后把结果合并成一个 DataFrame

apply 函数

写法	方向	func 接收的数据	结果数量
df.apply(f, axis=0)	按列	每列 (Series)	列数个结果
df.apply(f, axis=1)	按行	每行 (Series)	行数个结果

- 基本语法（以下超纲，供有兴趣同学学习）

DataFrame.apply(func, axis=0, raw=False, result_type=None, args=(), **kwargs)

参数	含义	可选值	默认值
func	要应用的函数	函数名 或 lambda 表达式	必填
axis	应用方向	0 或 'index'（按列） 1 或 'columns'（按行）	0
raw	是否直接传入 NumPy 数组	True / False	False
result_type	结果返回格式	'expand', 'reduce', 'broadcast'	自动判断
args	传递给函数的额外位置参数	元组	()
**kwargs	传递给函数的额外关键字参数	键值对	无

频数统计

- `value_counts` 函数可对 Series 或 DataFrame 的某一列进行值频数统计。

```
❶ s = pd.Series(np.random.randint(0, 5, size=5))  
s  
  
0 1  
1 4  
2 0  
3 4  
4 4  
dtype: int64  
  
❷ s.value_counts()  
  
4 3  
1 1  
0 1  
dtype: int64
```

合并：concat、merge、join

- `pd.concat()`：pandas 的拼接函数；像叠积木，上下堆或左右拼，不关心匹配
- `pd.merge()`：pandas 的匹配函数；根据某一系列的值把两个表拼在一起
- `df.join()`：DataFrame 的索引匹配方法；`pd.merge()` 的简化版，默认用行索引做匹配

```

import pandas as pd

df1 = pd.DataFrame({'A': [1, 2]}, index=[1, 2])
df2 = pd.DataFrame({'B': [3, 4]}, index=[2, 3])
print("df1: ")
print(df1)
print("\ndf2: ")
print(df2)

print("\n" + "=" * 40)
print("1. pd.concat([df1, df2]) —— 上下拼接")
print(pd.concat([df1, df2]))

print("\n" + "=" * 40)
print("2. pd.merge(df1, df2, left_index=True, right_index=True)"
)
print(" —— 按索引匹配 (保留交集) ")
print(pd.merge(df1, df2, left_index=True, right_index=True))

print("\n" + "=" * 40)
print("3. df1.join(df2) —— 按索引匹配 (保留左表全部) ")
print(df1.join(df2))

```

```
df1:
```

```

A
1 1
2 2

```

```
df2:
```

```

B
2 3
3 4

```

```
=====
```

```
1. pd.concat([df1, df2]) —— 上下拼接
```

```

A  B
1  1.0 NaN
2  2.0 NaN
2  NaN 3.0
3  NaN 4.0

```

```
=====
```

```
2. pd.merge(df1, df2, left_index=True, right_index=True)
—— 按索引匹配 (保留交集)
```

```

A  B
2  2 3

```

```
=====
```

```
3. df1.join(df2) —— 按索引匹配 (保留左表全部)
```

```

A  B
1  1 NaN
2  2 3.0

```

合并 concat

- 可用 concat 连接 pandas 对象：确定按某个轴进行连接（可横向可纵向），也可指定连接方法。concat 的参数如表所示。

方法	意义
objs	合并的对象集合，可以是 Series、DataFrame
axis	合并方法。默认 0 表示纵向，1 表示横向
join	默认 outer 并集，inner 交集，只有这两种
join_axes	按哪些对象的索引保存
ignore_index	默认 False 忽略，是否忽略原 index
keys	为原始 DataFrame 添加一个键，默认无

① `df = pd.DataFrame(np.random.randn(5,4))`
`df`

	0	1	2	3
0	0.407632	-0.912611	-0.348732	-0.570476
1	-1.257338	-0.476574	-0.379637	-0.312108
2	0.647044	1.612316	2.045953	0.304050
3	-0.265805	0.719232	-1.480685	-1.921071
4	1.559480	2.015801	1.325865	0.022508

② `pd.concat([df[:2], df[2:4], df[4:]])`

	0	1	2	3
0	0.407632	-0.912611	-0.348732	-0.570476
1	-1.257338	-0.476574	-0.379637	-0.312108
2	0.647044	1.612316	2.045953	0.304050
3	-0.265805	0.719232	-1.480685	-1.921071
4	1.559480	2.015801	1.325865	0.022508

合并 concat

语法结构:

```
pd.concat(objs, axis=0, join='outer', ignore_index=False)
```

```
import pandas as pd
```

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
```

```
df2 = pd.DataFrame({'A': [5, 6], 'C': [7, 8]})
```

```
print("df1: ")
```

```
print(df1)
```

```
print("\ndf2: ")
```

```
print(df2)
```

```
# 上下拼接
```

```
print("\npd.concat([df1, df2]): ")
```

```
print(pd.concat([df1, df2]))
```

```
# 左右拼接
```

```
print("\npd.concat([df1, df2], axis=1): ")
```

```
print(pd.concat([df1, df2], axis=1))
```

```
df1:
```

```
  A  B
```

```
0  1  3
```

```
1  2  4
```

```
df2:
```

```
  A  C
```

```
0  5  7
```

```
1  6  8
```

```
pd.concat([df1, df2]):
```

```
  A   B   C
```

```
0  1  3.0 NaN
```

```
1  2  4.0 NaN
```

```
0  5 NaN  7.0
```

```
1  6 NaN  8.0
```

```
pd.concat([df1, df2], axis=1):
```

```
  A  B  A  C
```

```
0  1  3  5  7
```

```
1  2  4  6  8
```

参数	含义	默认值
objs	要拼接的 DataFrame/Series 列表	必填
axis	拼接方向: 0=上下堆叠, 1=左右并排	0
join	索引合并方式: 'outer'=保留全部, 'inner'=只保留共有	'outer'
ignore_index	是否重置索引: True=生成新索引, False=保留原索引	False

合并 merge

- **merge** 函数通过一个或多个键将数据集的行连接起来。针对同一个主键存在的两张包含不同特征的表，通过主键的连接，将两张表进行合并。合并之后，两张表的行数不增加，列数是两张表的列数之和。

方法	意义
how	数据融合的方法，存在不重合的键，方式：inner、outer、left、right
on	用来对齐的列名，一定要保证左表和右表存在相同的列名
left_on	左表对齐的列，可以是列名，也可以是 DataFrame 同长度的 arrays
right_on	右表对齐的列，可以是列名
left_index	将左表的 index 用作连接键
right_index	将右表的 index 用作连接键
suffixes	左右对象中存在重名列，结果用后缀名区分
copy	默认True。将数据复制到数据结构中，设置为 False 提高性能

合并 merge

```
❶ df1 = pd.DataFrame({'key': ['one', 'two', 'two'], 'data1': np.arange(3)})  
df2 = pd.DataFrame({'key': ['one', 'three', 'three'], 'data2': np.arange(3)})  
df3 = pd.merge(df1, df2)  
print(df1)
```

```
   key  data1  
0  one     0  
1  two     1  
2  two     2
```

```
❷ print(df2)
```

```
   key  data2  
0  one     0  
1 three     1  
2 three     2
```

```
❸ print(df3)
```

```
   key  data1  data2  
0  one     0     0
```

```
❹ df4 = pd.merge(df1, df2, how='left')  
print(df4)
```

```
   key  data1  data2  
0  one     0     0.0  
1  two     1     NaN  
2  two     2     NaN
```

合并 merge

语法结构:

```
pd.merge(left, right, on=None, how='inner')
```

参数	含义	默认值
left	左表	必填
right	右表	必填
on	按哪一列匹配	None (自动找同名列)
how	匹配: 'inner'、'outer'、'left'、'right'	'inner'

```
import pandas as pd
```

```
df1 = pd.DataFrame({'ID': [1, 2, 3], '姓名': ['张三', '李四', '王五']})
df2 = pd.DataFrame({'ID': [2, 3, 4], '成绩': [85, 90, 88]})
```

```
print("df1: ")
print(df1)
print("\ndf2: ")
print(df2)
```

按 ID 列匹配 (默认 inner)

```
print("\npd.merge(df1, df2, on='ID'): ")
print(pd.merge(df1, df2, on='ID'))
```

左连接 (保留左表全部)

```
print("\npd.merge(df1, df2, on='ID', how='left'): ")
print(pd.merge(df1, df2, on='ID', how='left'))
```

```
df1:
   ID 姓名
0  1 张三
1  2 李四
2  3 王五
```

```
df2:
   ID 成绩
0  2  85
1  3  90
2  4  88
```

```
pd.merge(df1, df2, on='ID'):
   ID 姓名 成绩
0  2 李四  85
1  3 王五  90
```

```
pd.merge(df1, df2, on='ID', how='left'):
   ID 姓名 成绩
0  1 张三  NaN
1  2 李四  85.0
2  3 王五  90.0
```

合并 join

- `join` 方法将两个 DataFrame 中不同的列索引合并成一个 DataFrame。参数的意义与 `merge` 基本相同，只是 `join` 方法默认左外连接 `how=left`。

```
❶ df1 = pd.DataFrame({'A': ['A0', 'A1', 'A1'], 'B': ['B0', 'B1', 'B2']}, index=['K0', 'K1', 'K2'])
df2 = pd.DataFrame({'C': ['C1', 'C2', 'C3'], 'D': ['D0', 'D1', 'D2']}, index=['K0', 'K1', 'K3'])
df3 = df1.join(df2)
df4 = df1.join(df2, how='outer')
df5 = df1.join(df2, how='inner')
print(df3)
```

	A	B	C	D
K0	A0	B0	C1	D0
K1	A1	B1	C2	D1
K2	A1	B2	NaN	NaN

```
❷ print(df4)
```

	A	B	C	D
K0	A0	B0	C1	D0
K1	A1	B1	C2	D1
K2	A1	B2	NaN	NaN
K3	NaN	NaN	C3	D2

```
❸ print(df5)
```

	A	B	C	D
K0	A0	B0	C1	D0
K1	A1	B1	C2	D1

数据添加

语法结构:

```
df1.append(other, ignore_index=False)
```

- 可通过 `append` 方法将若干行添加到 `DataFrame` 后面

```
❶ df = pd.DataFrame(np.random.randn(3, 4), columns=['A', 'B', 'C', 'D'])  
df
```

	A	B	C	D
0	-0.388968	-0.815886	-0.584864	0.363678
1	-0.067046	-0.995409	0.296313	0.093199
2	1.232601	-0.353984	-0.509289	-1.585844

```
❷ s = df.iloc[1]  
s
```

```
A    -0.067046  
B    -0.995409  
C     0.296313  
D     0.093199  
Name: 1, dtype: float64
```

```
❸ df.append(s, ignore_index=True)
```

	A	B	C	D
0	-0.388968	-0.815886	-0.584864	0.363678
1	-0.067046	-0.995409	0.296313	0.093199
2	1.232601	-0.353984	-0.509289	-1.585844
3	-0.067046	-0.995409	0.296313	0.093199

参数	含义	默认值
other	要添加的 DataFrame、Series 或字典	必填
ignore_index	是否重置索引: True=生成新索引, False=保留原索引	False

⚠ 注意: Pandas 新版本 1.4.0+ 已弃用 `append()`, 推荐使用 `pd.concat()` 代替

分组

- Pandas 提供了一个高效灵活的 groupby 功能，能有效对数据集进行分组操作。groupby 操作，通常包括几个步骤：
 - (1) 分组：按照某些标准将数据分为不同的组。
 - (2) 应用：对每组数据分别执行一个函数。
 - (3) 组合：将结果组合到一个数据结构。

语法结构：
`df.groupby(by, axis=0)`

参数	含义	说明
by	按什么分组	列名、列名列表、Series、函数
axis	分组方向	0=按行分组，1=按列分组

分组后需跟聚合函数，
如 sum()、mean()、count()
才有实际意义

df.groupby('列名')['数值列'].mean()
按某列分组，再计算另一列的平
均值。分→组→算，三步完成。

```
import pandas as pd

df = pd.DataFrame({
    '班级': ['一班', '二班', '一班', '二班', '一班'],
    '姓名': ['张三', '李四', '王五', '赵六', '钱七'],
    '成绩': [85, 92, 78, 88, 95]
})

print("原始数据：")
print(df)

# 按班级分组，计算平均分
print("\n按班级分组，计算平均分：")
print(df.groupby('班级')['成绩'].mean())

# 按班级分组，计算总分
print("\n按班级分组，计算总分：")
print(df.groupby('班级')['成绩'].sum())

# 按班级分组，统计人数
print("\n按班级分组，统计人数：")
print(df.groupby('班级')['成绩'].count())
```

原始数据：

	班级	姓名	成绩
0	一班	张三	85
1	二班	李四	92
2	一班	王五	78
3	二班	赵六	88
4	一班	钱七	95

按班级分组，计算平均分：

班级

一班 86.0

二班 90.0

Name: 成绩, dtype: float64

按班级分组，计算总分：

班级

一班 258

二班 180

Name: 成绩, dtype: int64

按班级分组，统计人数：

班级

一班 3

二班 2

Name: 成绩, dtype: int64

分组

```
❶ df = pd.DataFrame({'A' : ['foo', 'bar', 'foo', 'bar'],  
                      'B' : ['one', 'one', 'two', 'three'],  
                      'C' : np.random.randn(4),  
                      'D' : np.random.randn(4)})
```

df

	A	B	C	D
0	foo	one	1.292104	-1.325384
1	bar	one	-1.050423	0.755551
2	foo	two	1.391699	-0.889404
3	bar	three	1.965823	-0.465339

```
❷ df.groupby('A').sum()
```

	C	D
A		
bar	0.915400	0.290212
foo	2.683803	-2.214789

```
❸ df.groupby(['A', 'B']).sum()
```

		C	D
A	B		
bar	one	-1.050423	0.755551
	three	1.965823	-0.465339
foo	one	1.292104	-1.325384
	two	1.391699	-0.889404

堆叠

语法结构：

```
df.stack(level=-1, dropna=True)
```

- `stack()` 方法将 DataFrame 的列“压缩”一个层级。

```
1 tuples = list(zip(*[['a', 'b', 'a', 'b'], ['one', 'two', 'three', 'four'])))
  index = pd.MultiIndex.from_tuples(tuples, names=['first', 'second'])
  df = pd.DataFrame(np.random.randn(4, 2), index=index, columns=['A', 'B'])
  df
```

参数		含义	默认值										
<table><tr><th>first</th><th>second</th></tr><tr><td>a</td><td>one</td></tr><tr><td>b</td><td>two</td></tr><tr><td>a</td><td>three</td></tr><tr><td>b</td><td>four</td></tr></table>	first	second	a	one	b	two	a	three	b	four	level	要堆叠的层级（用于多层索引）	-1（最内层）
first	second												
a	one												
b	two												
a	three												
b	four												
<table><tr><td>a</td><td>three</td></tr><tr><td>b</td><td>four</td></tr></table>	a	three	b	four	dropna	是否删除结果中的缺失值	True						
a	three												
b	four												

a	three	A	-1.187300
		B	0.092720
b	four	A	-0.467083
		B	0.781497

dtype: float64

堆叠

- `stack()` 方法将 `DataFrame` 的列“压缩”一个层级。

语法结构:

```
df.stack(level=-1, dropna=True)
```

参数	含义	默认值
level	要堆叠的层级（用于多层索引）	-1（最内层）
dropna	是否删除结果中的缺失值	True

```
import pandas as pd
```

```
df = pd.DataFrame({  
    '数学': [85, 92],  
    '语文': [90, 88]  
}, index=['张三', '李四'])
```

```
print("原始宽表: ")
```

```
print(df)
```

```
print("\nstack() 后（宽→长）: ")
```

```
print(df.stack())
```

原始宽表:

```
   数学  语文  
张三  85  90  
李四  92  88
```

stack() 后（宽→长）:

```
张三  数学  85  
      语文  90  
李四  数学  92  
      语文  88  
dtype: int64
```

`stack()` 把 `DataFrame` 的列标签压缩成行索引，让“宽表”变“长表”。

堆叠

- 对于一个“堆叠过的”DataFrame 或者 Series（拥有 MultiIndex 作为索引），`stack()` 的逆操作是 `unstack()`，默认反堆叠到上一个层级：

① `stacked.unstack()`

		A	B
first	second		
a	one	-1.400121	-0.275077
	three	-1.187300	0.092720
b	four	-0.467083	0.781497
	two	-0.987005	-1.722261

② `stacked.unstack(1)`

		second	four	one	three	two
first						
a	A	NaN	-1.400121	-1.18730	NaN	
	B	NaN	-0.275077	0.09272	NaN	
b	A	-0.467083	NaN	NaN	-0.987005	
	B	0.781497	NaN	NaN	-1.722261	

课堂练习4：学生成绩处理

- 你有两张表，完成以下任务：

	任务	使用的方法
1	将 stu 和 score 按"学号"合并	merge
2	将下面这个新生追加到 stu	concat
3	按班级分组，计算平均分	groupby
4	将成绩表变成"长表"（科目变成行）	stack
5	将 stu 和 score 按学号匹配（保留所有学生）	merge + how
6	给每行成绩加5分（福利分）	apply

```
import pandas as pd

# 表1: 学生信息
stu = pd.DataFrame({
    '学号': [101, 102, 103, 104],
    '姓名': ['张三', '李四', '王五', '赵六'],
    '班级': ['一班', '一班', '二班', '二班']
})

# 表2: 考试成绩
score = pd.DataFrame({
    '学号': [101, 102, 103, 104, 101, 102, 103, 104],
    '科目': ['数学', '数学', '数学', '数学', '语文', '语文', '语文', '语文'],
    '成绩': [85, 92, 78, 88, 90, 85, 88, 92]
})

new_stu = pd.DataFrame({'学号': [105], '姓名': ['钱七'], '班级': ['二班']})
```



12.3

汇总、计算和描述性统计

汇总、计算和描述性统计

- Pandas 拥有一套常用的数学和统计方法，但都是基于没有缺失数据的假设而构建的。

```
① df = pd.DataFrame([[9.72, 5.39], [9.62, 5.35], [9.53, 5.33]],  
index=['2020-10-16', '2020-10-15', '2020-10-14'],  
columns=['浦发银行', '民生银行'])  
df
```

	浦发银行	民生银行
2020-10-16	9.72	5.39
2020-10-15	9.62	5.35
2020-10-14	9.53	5.33

汇总、计算和描述性统计

- df 调用 DataFrame 的 `sum` 方法返回一个按列求和而得到的 Series。

❶ `df.sum()`

```
浦发银行    28.87  
民生银行    16.07  
dtype: float64
```

- 而如果传入参数 `axis=1`，则会按行进行求和运算，空值会被自动排除。

❷ `df.sum(axis=1)`

```
2020-10-16    15.11  
2020-10-15    14.97  
2020-10-14    14.86  
dtype: float64
```

汇总、计算和描述性统计

- 利用 `idxmax` 或 `idxmin` 方法，能找到达到最大值或最小值的索引。

❶ `df.idxmax()`

```
浦发银行 2020-10-16  
民生银行 2020-10-16  
dtype: object
```

- 想要求累加和，可以使用 `cumsum` 方法。

❷ `df.cumsum()`

	浦发银行	民生银行
2020-10-16	9.72	5.39
2020-10-15	19.34	10.74
2020-10-14	28.87	16.07

汇总、计算和描述性统计

- 如果想快速查看数据的统计汇总，可以直接采用 `describe` 方法。

❶ `df5.describe()`

	A	B	C	D
count	2	2	2	2
unique	2	2	2	2
top	A0	B1	C2	D0
freq	1	1	1	1

- 对于非数值型数据，`describe` 会产生另外一种汇总统计。

❷ `obj = Series(['a','a','b','c']*3)`
`obj.describe()`

```
count      12
unique       3
top         a
freq         6
dtype: object
```

汇总、计算和描述性统计

● 与统计描述有关的方法如表所示。

方法	意义
argmax、argmin	得到最大值和最小值的索引值
idxmax、idxmin	得到最大值和最小值索引的位置
count	对非空值的计数
describe	描述性统计
sum	求和
mean	求均值
median	求中位数
max、min	求最大值和最小值
quantile	n 分位数 (n 取 0 到 1)
mad	平均绝对离差
var	方差
std	标准差
skew	偏度 (三阶矩)
kurt	峰度 (四阶矩)
cumsum	累计和
cummax、cummin	累计最大值和累计最小值
comprod	累计积
diff	一阶差分
pct_change	百分数变化

汇总、计算和描述性统计

基本语法结构:

df.方法名(axis=0, skipna=True)

参数	含义	默认值
axis	方向: 0=按列, 1=按行	0
skipna	是否跳过缺失值	True

1. 基础统计 (返回单个值)

方法	含义	示例
count()	非空值个数	df['成绩'].count()
sum()	求和	df['成绩'].sum()
mean()	平均值	df['成绩'].mean()
median()	中位数	df['成绩'].median()
max()	最大值	df['成绩'].max()
min()	最小值	df['成绩'].min()
std()	标准差	df['成绩'].std()
var()	方差	df['成绩'].var()

汇总、计算和描述性统计

2. 位置统计（返回索引位置）

方法	含义	示例
<code>idxmax()</code>	最大值所在索引	<code>df['成绩'].idxmax()</code>
<code>idxmin()</code>	最小值所在索引	<code>df['成绩'].idxmin()</code>

3. 累计计算（返回同长度 Series）

方法	含义	示例
<code>cumsum()</code>	累计和	<code>df['成绩'].cumsum()</code>
<code>cummax()</code>	累计最大值	<code>df['成绩'].cummax()</code>
<code>cummin()</code>	累计最小值	<code>df['成绩'].cummin()</code>
<code>cumprod()</code>	累计积	<code>df['成绩'].cumprod()</code>

汇总、计算和描述性统计

4. 差异与变化

方法	含义	示例
diff()	一阶差分（与 前一行差值）	df['成绩'].diff()
pct_change()	百分比变化	df['成绩'].pct_change()

5. 分布形态

方法	含义	示例
quantile(q)	分位数（q=0~1）	df['成绩'].quantile(0.75)
skew()	偏度（对称性）	df['成绩'].skew()
kurt()	峰度（陡峭程度）	df['成绩'].kurt()
mad()	平均绝对离差	df['成绩'].mad()

6. 综合描述

方法	含义	示例
describe()	一次性输出多个统计值	df['成绩'].describe()

课堂练习5：成绩统计

- 已有学生成绩表，请使用 Pandas 统计方法完成以下任务：

题号	任务	使用的方法
1	统计学生人数和总分	count、sum
2	计算平均分和中位数	mean、median
3	找出最高分和最低分	max、min
4	找出最高分学生的姓名	idxmax
5	计算标准差和方差	std、var
6	计算成绩的累计和与累计最大值	cumsum、cummax
7	计算与前一名的成绩差和百分比变化	diff、pct_change
8	计算75%分位数、偏度和峰度	quantile、skew、kurt
9	一次性查看所有统计信息	describe

```
import pandas as pd

df = pd.DataFrame({
    '姓名': ['张三', '李四', '王五', '赵六', '钱七'],
    '成绩': [85, 92, 78, 88, 95]
})
```



12.4

案例

案例

【例】与书本不同，本节利用tushare读取招商银行股票数据，并进行Pandas基本操作。tushare为金融数据分析提供便捷、快速的接口，与投研和量化策略无缝对接。数据丰富，拥有丰富的数据内容，如股票、基金、期货、数字货币等行情数据，公司财务、基金经理等基本面数据。tushare是第三方库，因此，在首次使用时，需要通过pip安装。

```
① import pandas as pd
import tushare as ts
import datetime
ZSYH = ts.get_hist_data('600036', '2020-08-31', '2020-09-11')
ZSYH.head(3)
```

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover
date														
2020-09-11	37.87	37.88	37.24	37.02	553105.12	-0.46	-1.22	37.384	37.301	37.611	601673.64	683457.92	715096.09	0.27
2020-09-10	37.88	37.91	37.70	37.37	550146.06	0.10	0.27	37.296	37.422	37.649	588140.06	746441.58	720973.70	0.27
2020-09-09	37.27	37.95	37.60	37.16	605433.31	0.01	0.03	37.152	37.342	37.630	606206.76	748908.48	725490.75	0.29

```
② ZSYH.tail(3)
```

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover
date														
2020-09-02	37.49	37.58	37.20	36.80	797221.25	-0.22	-0.59	37.532	37.526	37.640	891610.20	754694.89	876003.27	0.39
2020-09-01	37.46	37.70	37.42	37.01	712759.38	-0.27	-0.72	37.538	37.641	37.595	857566.00	731366.46	889175.19	0.35
2020-08-31	38.50	39.00	37.69	37.55	1190313.62	-0.76	-1.98	37.570	37.784	37.558	824063.34	717456.58	971257.42	0.58

案例

❶ `ZSYH.index = pd.to_datetime(ZSYH.index) #将索引转换为日期索引`
`ZSYH.index`

```
DatetimeIndex(['2020-09-11', '2020-09-10', '2020-09-09', '2020-09-08', '2020-09-07', '2020-09-04', '2020-09-03',  
              '2020-09-02', '2020-09-01', '2020-08-31'],  
              dtype='datetime64[ns]', name='date', freq=None)
```

❷ `ZSYH['close'] #查看收盘价`

```
date  
2020-09-11    37.24  
2020-09-10    37.70  
2020-09-09    37.60  
2020-09-08    37.59  
2020-09-07    36.79  
2020-09-04    36.80  
2020-09-03    36.98  
2020-09-02    37.20  
2020-09-01    37.42  
2020-08-31    37.69  
Name: close, dtype: float64
```

案例

① `ZSYH[ZSYH.close>37]` #查看收盘价大于37元的交易信息

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover
date														
2020-09-11	37.87	37.88	37.24	37.02	553105.12	-0.46	-1.22	37.384	37.301	37.611	601673.64	683457.92	715096.09	0.27
2020-09-10	37.88	37.91	37.70	37.37	550146.06	0.10	0.27	37.296	37.422	37.649	588140.06	746441.58	720973.70	0.27
2020-09-09	37.27	37.95	37.60	37.16	605433.31	0.01	0.03	37.152	37.342	37.630	606206.76	748908.48	725490.75	0.29
2020-09-08	37.18	37.62	37.59	37.02	683544.94	0.80	2.17	37.072	37.305	37.640	644564.34	751065.17	741720.93	0.33
2020-09-02	37.49	37.58	37.20	36.80	797221.25	-0.22	-0.59	37.532	37.526	37.640	891610.20	754694.89	876003.27	0.39
2020-09-01	37.46	37.70	37.42	37.01	712759.38	-0.27	-0.72	37.538	37.641	37.595	857566.00	731366.46	889175.19	0.35
2020-08-31	38.50	39.00	37.69	37.55	1190313.62	-0.76	-1.98	37.570	37.784	37.558	824063.34	717456.58	971257.42	0.58

② `ZSYH['diff'] = ZSYH['close'] - ZSYH['open']` #增加收盘价与开盘价差值列
`ZSYH.head(3)`

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover	diff
date															
2020-09-11	37.87	37.88	37.24	37.02	553105.12	-0.46	-1.22	37.384	37.301	37.611	601673.64	683457.92	715096.09	0.27	-0.63
2020-09-10	37.88	37.91	37.70	37.37	550146.06	0.10	0.27	37.296	37.422	37.649	588140.06	746441.58	720973.70	0.27	-0.18
2020-09-09	37.27	37.95	37.60	37.16	605433.31	0.01	0.03	37.152	37.342	37.630	606206.76	748908.48	725490.75	0.29	0.33

案例

```
❶ def func(x):  
    return x.max()-x.min()  
ZSYH.apply(func) #应用自定义函数
```

```
open          1.870  
high          2.110  
close         0.910  
low           1.200  
volume        704876.400  
price_change  1.560  
p_change      4.150  
ma5           0.532  
ma10          0.483  
ma20          0.091  
v_ma5         316603.050  
v_ma10        71236.970  
v_ma20        256161.330  
turnover      0.340  
diff          1.220  
dtype: float64
```

案例

```
❶ pieces = [ZSYH[:3],ZSYH[3:6],ZSYH[6:]]  
pd.concat(pieces) # 连接
```

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover	diff
date															
2020-09-11	37.87	37.88	37.24	37.02	553105.12	-0.46	-1.22	37.384	37.301	37.611	601673.64	683457.92	715096.09	0.27	-0.63
2020-09-10	37.88	37.91	37.70	37.37	550146.06	0.10	0.27	37.296	37.422	37.649	588140.06	746441.58	720973.70	0.27	-0.18
2020-09-09	37.27	37.95	37.60	37.16	605433.31	0.01	0.03	37.152	37.342	37.630	606206.76	748908.48	725490.75	0.29	0.33
2020-09-08	37.18	37.62	37.59	37.02	683544.94	0.80	2.17	37.072	37.305	37.640	644564.34	751065.17	741720.93	0.33	0.41
2020-09-07	36.80	37.58	36.79	36.69	616138.75	-0.01	-0.03	37.038	37.304	37.637	650407.23	737235.28	794362.82	0.30	-0.01
2020-09-04	36.63	36.89	36.80	36.35	485437.22	-0.18	-0.49	37.218	37.375	37.648	765242.21	737720.30	824779.96	0.24	0.17
2020-09-03	37.12	37.43	36.98	36.76	640479.56	-0.22	-0.59	37.548	37.472	37.638	904743.11	745302.03	845405.80	0.31	-0.14
2020-09-02	37.49	37.58	37.20	36.80	797221.25	-0.22	-0.59	37.532	37.526	37.640	891610.20	754694.89	876003.27	0.39	-0.29
2020-09-01	37.46	37.70	37.42	37.01	712759.38	-0.27	-0.72	37.538	37.641	37.595	857566.00	731366.46	889175.19	0.35	-0.04
2020-08-31	38.50	39.00	37.69	37.55	1190313.62	-0.76	-1.98	37.570	37.784	37.558	824063.34	717456.58	971257.42	0.58	-0.81

案例

```
① dates = pd.date_range('2020-8-30', periods=15, freq='D')
ZSYH = ZSYH.reindex(dates, method='ffill') #重新索引
ZSYH.tail()
```

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover	diff
2020-09-09	37.27	37.95	37.60	37.16	605433.31	0.01	0.03	37.152	37.342	37.630	606206.76	748908.48	725490.75	0.29	0.33
2020-09-10	37.88	37.91	37.70	37.37	550146.06	0.10	0.27	37.296	37.422	37.649	588140.06	746441.58	720973.70	0.27	-0.18
2020-09-11	37.87	37.88	37.24	37.02	553105.12	-0.46	-1.22	37.384	37.301	37.611	601673.64	683457.92	715096.09	0.27	-0.63
2020-09-12	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2020-09-13	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
② ZSYH.dropna(inplace=True) # 删除空值行
ZSYH.tail()
```

	open	high	close	low	volume	price_change	p_change	ma5	ma10	ma20	v_ma5	v_ma10	v_ma20	turnover	diff
2020-09-07	36.80	37.58	36.79	36.69	616138.75	-0.01	-0.03	37.038	37.304	37.637	650407.23	737235.28	794362.82	0.30	-0.01
2020-09-08	37.18	37.62	37.59	37.02	683544.94	0.80	2.17	37.072	37.305	37.640	644564.34	751065.17	741720.93	0.33	0.41
2020-09-09	37.27	37.95	37.60	37.16	605433.31	0.01	0.03	37.152	37.342	37.630	606206.76	748908.48	725490.75	0.29	0.33
2020-09-10	37.88	37.91	37.70	37.37	550146.06	0.10	0.27	37.296	37.422	37.649	588140.06	746441.58	720973.70	0.27	-0.18
2020-09-11	37.87	37.88	37.24	37.02	553105.12	-0.46	-1.22	37.384	37.301	37.611	601673.64	683457.92	715096.09	0.27	-0.63

Happy coding! 🐍



综合练习6：电商订单分析

现有订单表和商品表，请完成分析任务：

题号	任务	使用的方法
1	将订单表和商品表按商品ID合并	merge
2	计算每笔订单的金额（数量×单价）	apply
3	找出金额最高的订单号	idxmax
4	按日期分组，计算每日总销售额	groupby
5	统计各商品被购买的次数	value_counts
6	将下面两天的新订单追加到原表	concat
7	查看金额的统计信息	describe

```
new_orders = pd.DataFrame({
    '订单号': [106, 107],
    '日期': ['2024-01-03', '2024-01-04'],
    '商品ID': [1, 4],
    '数量': [1, 3]
})
```

```
import pandas as pd
```

```
# 订单表
```

```
orders = pd.DataFrame({
    '订单号': [101, 102, 103, 104, 105],
    '日期': ['2024-01-01', '2024-01-01', '2024-01-02', '2024-01-02', '2024-01-03'],
    '商品ID': [1, 2, 1, 3, 2],
    '数量': [2, 1, 3, 5, 2]
})
```

```
# 商品表
```

```
products = pd.DataFrame({
    '商品ID': [1, 2, 3, 4],
    '商品名': ['手机', '耳机', '充电器', '数据线'],
    '单价': [2000, 299, 89, 49]
})
```

综合练习7：健身房会员分析

现有会员签到表和课程表，请完成分析任务：

题号	任务	使用的方法
1	将签到表和课程表按课程ID合并	merge
2	计算每位会员的总锻炼时长	groupby
3	找出锻炼时长最长的会员	idxmax
4	统计各课程被选择的次数	value_counts
5	按日期分组，统计每日签到人数	groupby + nunique
6	找出每位会员最喜欢的课程（上课次数最多）	groupby + idxmax
7	查看时长的统计摘要	describe

```
import pandas as pd
```

```
# 会员签到表
```

```
checkins = pd.DataFrame({  
    '会员名': ['小明', '小红', '小明', '小刚', '小红', '小丽', '小明', '小刚'],  
    '日期': ['2024-01-01', '2024-01-01', '2024-01-02', '2024-01-02',  
             '2024-01-03', '2024-01-03', '2024-01-04', '2024-01-04'],  
    '课程ID': [1, 2, 1, 3, 2, 1, 3, 1],  
    '时长': [60, 45, 60, 30, 45, 60, 30, 60]  
})
```

```
# 课程表
```

```
courses = pd.DataFrame({  
    '课程ID': [1, 2, 3],  
    '课程名': ['瑜伽', '动感单车', '力量训练'],  
    '教练': ['李老师', '王教练', '张教练']  
})
```